
Multiverse Singularity Optimization

Release 1.9.0

Multiverse Computing

May 09, 2024

CONTENTS

Welcome to the Singularity Optimization (aka SOP) documentation! SOP is a Mathematical Optimization package. It is a python library that allows to implement, solve, and analyze generic optimization problems using quantum or quantum-inspired solvers.

CONTENT:

1.1 Installation

Using pip we can install or update the package directly from the whl file:

```
pip install multiverse_singularity_optimization-1.9.0-py3-none-any.whl
```

1.1.1 Authentication

To use the package you should provide your token to the package. First, using the [login page](#), you access your user dashboard to get your token. Then you can use one of the methods listed below to use your token in the package. If you want to run optimization tasks on D-Wave Systems Quantum computers, you have to bring your own token for dwave.

1. In environment variables named SINGULARITY_TOKEN and DWAVE_API_TOKEN.
2. In a file named singularity_config.json in the current working directory using the key token and dwave_api_token.
3. Like the second option but put the file in the *home directory*.

Listing 1: singularity_config.json example

```
{
  "token": "your_token_here",
  "dwave_api_token": "your_token_here"
  "ssl_verify": "True"
}
```

Note: Precedence of the tokens follows the order as they are listed (i.e. environment variables will override other configurations)

1.1.2 SSL Verification

For constrained environments that filter and change SSL signatures you may want to disable SSL verification. You can do this by setting the ssl_verify option to "False" in singularity_config.json, or by setting the environment variable SINGULARITY_SSL_VERIFY to False.

Note: the underlying python urllib will log warnings of unverified SSL usage. For the best security adding certificate verification is strongly advised. See: <https://urllib3.readthedocs.io/en/latest/advanced-usage.html#tls-warnings>.

1.2 Basic Usage

1.2.1 Knapsack Problem

This is an example using Singularity Optimization to solve the knapsack problem.

Knapsack Problem: Given a set of items, each with a weight, a value, and an extra profit that can be earned if two items are both selected, determine which items to include in a collection so as to maximize the overall profit without exceeding the capacity of the knapsack. In this formulation, we seek to find the collection of items whose total weight is less than or equal to the knapsack capacity while maximizing the value of the selection.

Think, for example, of packing a shipping container with goods where the container has an overall weight capacity. Each good has its own inherent value and possibly additional value if paired with some other good (a pair of matching shoes has more value than the sum of each individual shoe).

Approach: In our problem we are trying to *maximize* profit. In an optimization problem where the sense (i.e., direction) is maximization, we say generally that we are maximizing *utility*. Conversely, where the sense is minimization, we say that we are minimizing *cost*. In order to find the optimal solution we will write a *utility function* such that the optimal configuration has the highest utility. We will then run this function through the D-Wave exact solver accessible using Singularity. This method exhaustively searches over all possible configurations and sorts them to select the lowest-cost (i.e., optimal) configuration.

(Note that Singularity always optimizes a *cost function*. When given a utility function, Singularity converts to a minimization problem by multiplying the function by negative one. In general, both a utility or a cost function can be referred to as the *objective function*.)

The D-Wave solver requires that the objective function is at most quadratic in its problem variables. For a small problem size (fewer than 18 variables), the exact solver is sufficient to find the configuration which maximizes the utility (i.e., value of items assigned to the knapsack) subject to the capacity constraint. Scaling to larger problems will be too much for the exact solver to handle. Instead, these could be solved by sampling, for example, the D-Wave Quantum Processing Unit.

1.2.2 Imports

We can import singularity optimization like this

```
>>> import singularity.optimization as sop
```

which helps prevent name collisions in the current namespace.

Then we use these classes to model and solve an optimization problem: the `sop.Constraint` class used to represent a constraint for the problem; the `sop.Model` class which combines the objective and any constraints into a single model; the `sop.Objective` class to build the utility function; the `sop.Variable` class used to represent the problem variables (`sop.INTEGER` to indicate integer variables: `sop.Variable("b")` creates a binary variable `b` while `sop.Variable("i", sop.INTEGER, bounds=(0,10))` creates an integer variable `i`, $0 \leq i \leq 10$).

Variables are binary by default, meaning that they may take *0* or *1* as values. A *1* indicates that an object has been selected to the knapsack, *0* otherwise.

1.2.3 Objective Function

Next comes time to build the objective function. Our objective is to maximize the value in the knapsack (the capacity constraint comes in later). In terms of the assignment variables $x[i]$, the objective function is

$$\text{Total Value} = \sum_i \text{Item-Values}[i] \cdot x[i] + \sum_{i < j} \text{Pair-Values}[(i, j)] \cdot x[i]x[j],$$

Write the *total_value* expression using the *coefficients* and the *variables* much like you would write a mathematical expression with pen and paper:

```
>>> x1 = sop.Variable("x1")
>>> x2 = sop.Variable("x2")
>>> x3 = sop.Variable("x3")
>>> total_value = 2 * x1 + 3 * x2 + 6 * x3 + 2 * x1 * x2 + 4 * x1 * x3 + 3 * x2 * x3
```

The *total_value* expression now gets stored in an *Objective* object with an indication of the sense (maximize in this case):

```
>>> objective = sop.Objective(total_value, "maximize")
```

1.2.4 Constraint

Now comes the time to implement the capacity constraint which indicates the maximum total weight of items placed in the knapsack,

$$\sum_i \text{Weights}[i] \cdot x[i] \leq \text{Capacity}.$$

To do this, write an expression for the left-hand-side of the constraint (similar to how the *total_value* was done above). This then is stored in a *Constraint* object along with the operator (less-than-or-equal-to), a name for the constraint, and the penalty strength (a multiplier which penalizes infeasible solutions).

We could set the penalty as high as we like and the exact solver would still be able to discern between feasible solutions. However, too high of a penalty strength could make it hard for a different sampler to distinguish between feasible solutions when their differences are small relative to the full scale of the model. Selecting the proper penalty strength requires that it be large enough so that all infeasible solutions have a lower utility (or higher cost for a minimization problem) than the optimal feasible solution, but small enough penalty so as to maintain adequate resolution between the feasible solutions. This action of finding the ideal penalty strength is often achieved empirically.

For this exercise, we suggest *penalty_strength* = 10 and *capacity* = 10:

```
>>> penalty_strength = 10
>>> capacity = 10
>>> total_weight = 4 * x1 + 5 * x2 + 1 * x3
>>> capacity_constraint = sop.Constraint(
...     lhs=total_weight,
...     operator="<=",
...     rhs=capacity,
...     penalty_strength=penalty_strength,
...     name="capacity_constraint",
... )
```

1.2.5 Optimization

Now we bundle the objective and a list of all problem constraints (one in this case) and store them in a *Model* object.

```
>>> knapsack = sop.Model(objective)
>>> knapsack.add_constraint(capacity_constraint)
```

or

```
>>> knapsack = sop.Model(objective, [capacity_constraint])
```

With the model ready to go, all we need to do is select the optimizer we wish to use and run the model through the optimization (might take a few seconds).

For instance, we could select the *qpu* or *hybrid* D-Wave samplers. The *num_solutions* argument indicates how many samples are to be taken, and after running, the result object will contain the best found solution in *values*. We can access all the obtained samples in *samples* which can be useful when using probabilistic solvers such as quantum machines. For other solvers, *samples* might contain similar information such as other solutions found along the way.

In this example, we will be using the D-Wave *simulated_annealing* solver. After running this solver with *num_solutions* equal to 5, in the *result* variable we will be able to see the best found solution and the value of the objective function at this solution; additionally, the *samples* attribute will contain the unique solutions found in the 5 shots.

```
>>> result = knapsack.optimize(solver="simulated_annealing", num_solutions=5)
```

Optimization jobs do not run indefinitely, we can set the maximum number of seconds via the *timeout* parameter (*int*, 1800 by default= 30 minutes). For example, limiting the knapsack problem to run in at most 60 seconds:

```
>>> result = knapsack.optimize(solver="simulated_annealing", num_solutions=5, timeout=60)
```

If the optimization time exceeds the limit, then a *SOPTimeoutError* will be raised.

1.2.6 Results

The results of the optimization are returned as an *OptimizationResult* object:

- *result.values* will contain the best found solution as a dictionary where the keys are the variables defined in the model and the keys are the values found.
- *result.objective_value* will contain the value of the objective function evaluated at the best found solution.
- *result.samples* gives a list of all unique samples, each as a *Solution* object (which includes relevant attributes like *cost* and *variables*). When using the exact solver, this is a list of the best results that is *num_solutions* long.

```
>>> result.values
{x1: 1, x2: 0, x3: 1}
>>> result.objective_value
-12.0
>>> result.samples
[Solution(variables={x1: 1, x2: 0, x3: 1}, cost=-12.0, num_occurrences=1), Solution(variables={x1: 0,
↪x2: 0, x3: 1}, cost=-6.0, num_occurrences=1), Solution(variables={x1: 0, x2: 1, x3: 0}, cost=-3.0,
↪num_occurrences=1), Solution(variables={x1: 0, x2: 1, x3: 1}, cost=-2.0, num_occurrences=1),
↪Solution(variables={x1: 1, x2: 0, x3: 0}, cost=-2.0, num_occurrences=1)]
```

1.2.7 Run problem from the json payload

You can run the problem from the json payload of the model in 2 ways:

1- From the terminal:

```
python -m singularity.optimization path/to/json_payload
```

2- Load the model in your python code:

```
import singularity.optimization as sop
model, payload = sop.Model.from_json("path/to/json") # model object, payload contains the
↳ hyperparameters and solver options sent from the client.
result = model.optimize(**payload)
```

```
import singularity.optimization as sop
model, _ = sop.Model.from_json("path/to/json")
result = model.optimize(solver="classical", num_solutions=5)
```

1.3 Solvers

In this section we describe the solvers available through singularity.

Note: Not all input parameters are listed, links to solver specific parameter lists are provided for D-Wave based solvers.

1.3.1 D-Wave Solvers

D-Wave provides a variety of samplers, SOP has migrated them as separate solvers.

D-Wave Exact

The **D-Wave exact** solver is a simple solver which is useful for testing and debugging purposes. It is an exhaustive solver, meaning that it samples the entire state space to find the optimal solution.

Note: The solver becomes slow for problems with 18 or more variables.

Solver Category

Classical.

Solver Suitability

General QP problem.

Parameters

- *num_solutions*: The best *num_solutions* will be returned by the solver.
- *dwave_api_token*: Your dwave api token (if this parameter is not set, the token will be retrieved from the DWAVE_API_TOKEN environment variable or the singularity_config.json file).

Example

Below we solve the knapsack problem defined in the *Basic Usage* section. In this example we set *num_solutions*=8, thus the solver will return all the possible solutions.

```
>>> result = knapsack.optimize(solver="dwave_exact", num_solutions=8, dwave_api_token="your_
↪token_here")
```

D-Wave QPU

The D-Wave QPU solver is a purely quantum solver which uses the D-Wave systems Advantage QPU. The the Advantage QPU uses quantum annealing to find the optimal value of the cost function.

Solver Category

QPU.

Parameters

- *num_solutions*: The number of samples to be taken.
- *annealing_time*: The quantum annealing time in microseconds.
- *dwave_api_token*: Your dwave api token (if this parameter is not set, the token will be retrieved from the DWAVE_API_TOKEN environment variable or the singularity_config.json file).

For a full list of parameters see the [D-Wave documentation](#).

Example

Below we use the D-Wave qpu solver to solve the knapsack problem by taking 50 samples.

```
>>> result = knapsack.optimize(solver="dwave_qpu", num_solutions=50, dwave_api_token="your_
↪token_here")
```

From the list in [D-Wave documentation](#), you can extend the annealing schedule of the QPU by

```
>>> # 100ts for first 70% of the annealing, another 100ts for the remaining 30%
>>> result = knapsack.optimize(solver="dwave_qpu", num_solutions=50, anneal_schedule=[[0, 0], [100,
↪ .7], [200, 1]])
```

The rest of parameters provided by D-Wave are passed as keyword arguments for both this and the D-Wave Hybrid solver.

D-Wave Hybrid

Singularity also provides access to D-Wave [Leaps hybrid solver](#). This is a state-of-the-art hybrid solver which uses classical and quantum computing resources together an intelligent way. The D-Wave hybrid solver can deal with problems with up to 1,000,000 variables.

Solver Category

QPU.

Parameters

- *num_solutions*: Number of times to run the optimizer.
- *time_limit*: Maximum processing time for the optimizer D-Waves facilities in seconds; this time limit applies to each of the attempted runs (*num_solutions*) of the optimizer.
- *dwave_api_token*: Your dwave api token (if this parameter is not set, the token will be retrieved from the DWAVE_API_TOKEN environment variable or the singularity_config.json file).

For a full list of parameters see the [D-Wave documentation](#).

Example

Below we run the D-Wave hybrid solver 5 times on the knapsack problem with a limit of 5min. per run:

```
>>> result = knapsack.optimize(solver="dwave_hybrid", num_solutions=5, time_limit=300, dwave_
↪ api_token="your_token_here")
```

D-Wave Simulated Annealing

The Simulated Annealing is a classical meta-heuristic solver, it should perform better than the *D-Wave Exact* solver and it can be used as another classical optimizer to which you can compare the QPU based ones.

Solver Category

Classical.

Parameters

- *num_solutions*: The number of samples to be taken.
- *dwave_api_token*: Your dwave api token (if this parameter is not set, the token will be retrieved from the DWAVE_API_TOKEN environment variable or the singularity_config.json file).

For a full list of parameters see the [D-Wave documentation](#).

Example

Below we use the D-Wave Simulated Annealing solver to solve the knapsack problem by taking 50 samples.

```
>>> result = knapsack.optimize(solver="simulated_annealing", num_solutions=50, dwave_api_token=
↳ "your_token_here")
```

or with more parameters

```
>>> result = knapsack.optimize(
...     solver="simulated_annealing",
...     num_solutions=50,
...     beta_schedule_type="geometric",
...     seed=345,
...     initial_states_generator="random"
... )
```

1.3.2 Classical

Warning: The classical solver is scheduled for deprecation. Currently, requests to use the classical solver are being redirected to the D-Wave Simulated Annealing solver. Future releases will no longer support the classical solver.

The *classical* solver for general small to middle sized quadratic optimization problems, which is also useful for testing and debugging purposes. With respect to the size, the maximum number of variables allowed for the Classical solver is 999.

This solver returns only one feasible solution, however, it may not always be able to find it thus raising a *NoSolutionError* exception. If this happens, you can rerun the job with increased the exploration time by increasing the *nlp_maximum_iterations* parameter. Bear in mind that for unfeasible problems it will not be able to find an optimum.

As with other solvers, the classical solver runs entirely on the Singularity Optimization backend.

Solver Category

Classical.

Solver Suitability

General QP problem with less than 1000 variables.

Parameters

- *nlp_maximum_iterations*: the maximum number of iterations when trying to solve the quadratic part (nlp ~ non-linear-problem). Defaults to 50.

Example

```
>>> result = knapsack.optimize(solver="classical")
```

1.4 Reference

1.4.1 Modelling

```
class singularity.optimization.Variable(name: str, var_type:
    Optional[singularity.optimization.modelling.variable.VariableType] =
    VariableType.BINARY, bounds: Optional[Tuple[int, int]] = None)
```

Bases: symengine.lib.symengine_wrapper.Symbol

Represents a problem variable.

Parameters

- name (str) – The name of the variable.
- var_type (VariableType) – The type of the variable. Possible values are sop.BINARY or sop.INTEGER.
- bounds (Optional[Tuple[int, int]]) – The lower and upper bounds of the variable, in that order. It is required for integer variables.

Examples:

```
x1 = Variable("x1")
x2 = Variable("x2", sop.INTEGER, (0, 10))
```

property bounds: Tuple[int, int]

Variable bounds.

property var_type: singularity.optimization.modelling.variable.VariableType

Variable type.

property ub: int

Upper bound of the variable.

property lb: int

Lower bound of the variable.

singularity.optimization.SymbolicExpression

Represents an expression.

Example:

$$\text{expr} = 2 * x1^{**2} + x2$$

class singularity.optimization.Objective(*expression, sense*)

Bases: object

Represents the optimization objective.

Parameters

- expression (SymbolicExpression) – The expression to be minimized/maximized.
- sense (str) – The optimization sense. Possible values are maximize and minimize.

property expression: symengine.lib.symengine_wrapper.Expr

The expression.

property sense: str

Getter for protected attribute `_sense`.

class singularity.optimization.Constraint(*lhs, operator, rhs, penalty_strength, name*)

Bases: object

Represents a constraint for a problem.

Parameters

- lhs (SymbolicExpression) – The left-hand side of the constraint equation.
- operator (str) – The operator of the constraint equation.
- rhs (Union[float, SymbolicExpression]) – The right-hand side of the constraint equation.
- penalty_strength (float) – The penalty strength for the constraint.
- name (str) – The name of the constraint.
- inequality_precision (int) – inequality precision value

property lhs: symengine.lib.symengine_wrapper.Expr

Left hand side of constraint expression.

property rhs: Union[float, symengine.lib.symengine_wrapper.Expr]

Right hand side of constraint expression.

property penalty_strength: float

Penalty strength for the constraint.

as_tuple()

Represent the constraint as a tuple (*lb, expr, ub*) corresponding to $lb \leq expr \leq ub$

class singularity.optimization.Model(*objective, constraints*)

Bases: object

Defines an optimization model.

Parameters

- objective (Objective) – The objective.
- constraints (Optional[Collection[Constraint]]) – A list of constraints. (Optional)

property num_variables: int

Number of variables.

property num_constraints: int

Number of constraints.

property objective_type: str

Objective type.

property constraint_type: str

Constraint type.

property variable_type: str

Variable type.

property constraints: List[singularity.optimization.modelling.constraint.Constraint]

A list of constraints.

property variables: Set[singularity.optimization.modelling.variable.Variable]

The variables of the model.

property variable_lower_bounds: numpy.ndarray

A vector containing the lower bounds of variables.

property variable_upper_bounds: numpy.ndarray

A vector containing the upper bounds of variables.

property objective: Optional[singularity.optimization.modelling.objective.Objective]

Symbolic representation of the model objective.

add_constraint(*constraint*: singularity.optimization.modelling.constraint.Constraint) → None

Appends a constraint to the list of constraints.

add_constraints(*constraints*: Collection[singularity.optimization.modelling.constraint.Constraint]) → None

Appends a list of constraints to the list of constraints.

add_variable(*variable*: singularity.optimization.modelling.variable.Variable) → None

Adds a variable to the model.

add_variables(*variables*: Collection[singularity.optimization.modelling.variable.Variable]) → None

Adds a list of variables to the model.

classmethod from_quadratic_model(*linear_objective_vector*: Optional[scipy.sparse._csr.csr_matrix] = None, *quadratic_objective_matrix*: Optional[scipy.sparse._csr.csr_matrix] = None, *constant*: float = 0, *sense*: str = 'minimize', *variable_type*: str = 'integer', *linear_constraint_matrix*: Optional[scipy.sparse._csr.csr_matrix] = None, *constraint_lower_bounds*: Optional[numpy.ndarray] = None, *constraint_upper_bounds*: Optional[numpy.ndarray] = None, *variable_lower_bounds*: Optional[numpy.ndarray] = None, *variable_upper_bounds*: Optional[numpy.ndarray] = None, *penalty_strengths*: Optional[numpy.ndarray] = None) → singularity.optimization.modelling.model.Model

Creates a new model from data specifying a quadratic model.

The model is define as: $\min/\max 0.5 * x^T Q x + b^T x + c$ such that: $cl \leq Ax \leq cu, l \leq x \leq u$

Number of variables = N Number of constraints = M

Parameters

- linear_objective_vector (scipy.sparse.csr_matrix[1, N]) –
b.
- quadratic_objective_matrix (scipy.sparse.csr_matrix[N, N]) –
Q.
- constaint (float) –
c.
- sense (str) – minimize or maximize.
- variable_type (str) – binary or integer, if binary variables may be omitted.
- linear_constraint_matrix (scipy.sparse.csr_matrix[M, N]) –
A.
- constraint_lower_bounds (numpy.ndarray[M]) –
cl.
- constraint_upper_bounds (numpy.ndarray[M]) – cu.
- variable_lower_bounds (numpy.ndarray[N]) –
l.
- variable_upper_bounds (numpy.ndarray[N]) –
u.
- penalty_strengths (numpy.ndarray[M]) – Intial values for penalty strengths.

Returns A model.

classmethod from_qubo(*qubo*: Dict[Tuple[int, int], float], *offset*: float = 0, *sense*: str = 'minimize') →
singularity.optimization.modelling.model.Model

Creates a new model from a QUBO represented as a dictionary.

Parameters

- qubo – A dictionary where the keys denote the indices of a QUBO matrix and the values denote the coefficients.
- offset – The offset will be added to the objective.

Returns A model.

eval_constraints(*solution*: Union[tuple, numpy.ndarray,
Dict[singularity.optimization.modelling.variable.Variable, int]]) → Dict[str, bool]

Evaluates if constraints are respected given variable values in a solution.

eval_objective(*solution*: Union[tuple, numpy.ndarray,
Dict[singularity.optimization.modelling.variable.Variable, int]]) → float

Evaluates the models objective function at the provided solution point.

classmethod from_json(input_json: str | dict | io.TextIOWrapper) → Tuple[Model, dict]

Loads a job from JSON format into a *Model* object and optimizer kwargs.

Parameters input_json (str or dict or io.TextIOWrapper) – The JSON file path, JSON loaded as a dict, or file-like object containing the model data.

Returns

The created *Model* object. dict: A dictionary containing the optimizer kwargs of the optimize method,

including hyperparameters and solver options.

Return type *Model*

optimize(solver: str, num_solutions: int = 5, timeout: int = 1800, **solver_options) → Union[None, singularity.optimization.result.OptimizationResult]

Runs the optimization.

Parameters

- solver (str) – The solver. See *Solvers* for the list of possible solvers.
- num_solutions (int) – Number of solutions to return.
- timeout (int) – Maximum number of seconds for the optimization to take.
- **solver_options – Available options depend on *solver*, see *Solvers*.

Returns Optimization result, containing solutions.

Return type *OptimizationResult*

1.4.2 Optimization

class singularity.optimization.Solution

Bases: object

Solution data.

The data are immutable.

variables: Dict[singularity.optimization.modelling.variable.Variable, int]

Solutions variables and their values.

objective_value: float

The objective value of the solution

is_feasible: bool

Feasibility of the solution.

num_occurrences: Optional[int] = None

When the solutions is sampled from a solution set, the number of occurrences.

property cost

Solutions objective value.

sop.Solution: cost is deprecated and will be removed in version 2.0.0, use *objective_value* instead.

property `qpu_readout`

Solutions variables and their values.

sop.Solution: `qpu_readout` is deprecated and will be removed in version 2.0.0, use *variables* instead.

class `singularity.optimization.OptimizationResult`

Bases: `object`

Result of the optimization.

The data are immutable.

samples: `List[singularity.optimization.optimization.solution.Solution]`

A list of generated samples for probabilistic solvers. These samples might represent infeasible solutions.

runtime: `float`

The time in seconds it took the optimization job to finish.

details: `Optional[dict]`

Additional solver-dependent information about the optimization process.

values: `Optional[Dict[singularity.optimization.modelling.variable.Variable, float]] = None`

Values of the best solution found by the solver. Is `None` if no feasible solution could be found.

objective_value: `Optional[float] = None`

Value of the objective function at the found solution. Is `None` if no feasible solution could be found.

status: `Optional[str] = None`

Status of the optimization process.

sense: `str = 'minimize'`

The sense of the optimization problem

property `solutions`: `List[singularity.optimization.optimization.solution.Solution]`

Deprecated way to access the samples.

property `optimizer_info`: `dict`

Deprecated way to access the details.

property `best`: `singularity.optimization.optimization.solution.Solution`

Deprecated way to return best found solution (lowest cost).

property `worst`: `singularity.optimization.optimization.solution.Solution`

Worst found solution (highest cost). Its usage is deprecated.

1.5 Changelog

1.5.1 Release 1.9.0

Dependencies

- Deprecation notice: `tensor_network_experimental` and classical solvers are now deprecated.
- To use the package you should provide your token to the package. First, using the login page <https://singularity-common.singularity-quantum.com/dashboard>.

Bugs fixed

- Some hardcoded lines in the integration tests.
- Small architecture-related bug in dockerfiles.

Features added

- Integration tests compatibility running from SOP backend.
- Enhanced authentication methods.
- To use the package you should provide your token to the package. First, using the login page <https://singularity-common.singularity-quantum.com/dashboard>.

1.5.2 Release 1.8.17

Dependencies

Previous notices still apply:

- Note for developers: you can begin using the new Singularity tokens at <https://singularity-common.singularity-quantum.com/dashboard>.
- Deprecation notice: release 2.0.0 will no longer allow the use of old raw Cognito tokens in favour of the shorter Singularity tokens.

Incompatible changes

- D-Wave no longer runs multiple times, it returns the suited number of samples. This miss-feature ended up eating many peoples credits, so it was removed. Run the optimizer in a loop if you really intend multiple annealer runs.

Bugs fixed

- Bottleneck with dense problems fixed.

Features added

- Devs only: tabu and greedy are available.
- Multiverse Quantum Inspired solver now officially available.
- Improved constraint rounding, no rounding happens if not necessary.

1.5.3 Release 1.8.16

Dependencies

- In order to support Matrix inputs, SOP depends on numpy and scipy.

Previous notices still apply:

- Note for developers: you can begin using the new Singularity tokens at <https://singularity-common.singularity-quantum.com/dashboard>.
- Deprecation notice: release 2.0.0 will no longer allow the use of old raw Cognito tokens in favour of the shorter Singularity tokens.

Incompatible changes

- Going with the officially supported python versions, Python 3.7 is no longer supported. We encourage you to use SOP with a newer version. See <https://devguide.python.org/versions/> for the status of Python versions.
- Models created with `Model.from_qubo` and `Model.from_json` will now have their `objective` attribute set to `None`, to evaluate the objective value of a solution use `Model.eval_objective()`

Bugs fixed

- Performance bottlenecks in `Model.from_qubo` and `Model.from_json` have been alleviated
- Fixed bug whereby `OptimizationResult.best` did not return the best solution

Features added

- New method `Model.from_quadratic_model` to create a model from sparse matrix inputs.

1.5.4 Release 1.8.15

Dependencies

Previous notices still apply:

- Note for developers: you can begin using the new Singularity tokens at <https://singularity-common.singularity-quantum.com/dashboard>.
- Deprecation notice: release 2.0.0 will no longer allow the use of old raw Cognito tokens in favour of the shorter Singularity tokens.

Deprecated

- Old backend implementation of solvers will be removed in the next release.

Bugs fixed

- PO compatibility error fixed, instantiation of OptimizationResult was not compatible with PO. This results in better integration with PO.
- Integer constraints treatment has been fixed in the backend.
- Connection error when setting the SOP endpoint (i.e. point to a different environment); SINGULARITY_OPTIMIZATION_ENDPOINT environment variable.

Features added

- All solvers have been migrated to the new backend architecture.

1.5.5 Release 1.8.14

Dependencies

- Note for developers: you can begin using the new Singularity tokens at <https://singularity-common.singularity-quantum.com/dashboard>.
- Deprecation notice: release 2.0.0 will no longer allow the use of old raw Cognito tokens in favour of the shorter Singularity tokens.

Bugs fixed

- Authentication endpoint error fixed in the backend.

Features added

- Classical solver and TN Experimental Optimizer have been migrated to the new faster workflow.
- Experimental: you can change the SOP endpoint (i.e. point to a different environment) by setting the URL in the SINGULARITY_OPTIMIZATION_ENDPOINT environment variable.

1.5.6 Release 1.8.13

Dependencies

- No new dependencies.
- Note for developers: make sure you update the docker:dind images you are using as 2019 images are now EOL (End Of Life).

Bugs fixed

- Setting the initial state in the D-Wave solvers no longer raises an exception; this was also affecting Multiverse Hybrid.
- Regression fix: constraint bounds error when rounding them in the classical solver.
- Timeout handling cleanup.
- Constraint lower and upper bounds rounding.
- Documentation corrections.
- Gurobi solver misrouting backend error fixed.
- PO Classical and Multiverse Hybrid: fix inconsistent status for some convex problems (this is the long-standing kNotSet bug in the backend).

Features added

- Feasibility check for best solution (accessible in `OptimizationResult.values`).
- Standardized cost computation for gurob, D-Wave*, po_classical, exhaustive solvers. TODO: TN based solvers.
- Overall faster modelling.
- Allow for $Q=0$ in both TN Optimizers, previously limited to quadratic problems.
- TN Optimizer has a new parameter `initial_mps`, it can be either "random" (default) or "superposition".
- Gurobi solver handles integer variables

1.5.7 Release 1.8.12

Dependencies

- No new dependencies.
- Note for developers: make sure you update the docker:dind images you are using as 2019 images are now EOL.

Bugs fixed

- Setting the initial state in the D-Wave solvers no longer raises an exception; this was also affecting Multiverse Hybrid.
- Regression fix: constraint bounds error when rounding them in the classical solver.
- Timeout handling cleanup.
- Constraint lower and upper bounds rounding.
- Documentation corrections.
- Performance: `add_constraints` had accidental $O(n^2)$ complexity. Its now $O(n)$.
- D-Wave API tokens will not be sent for non D-Wave solvers.

Features added

- Allow for $Q=0$ in both TN Optimizers, previously limited to quadratic problems.
- TN Optimizer has a new parameter `initial_mps`, it can be either "random" (default) or "superposition".
- New `sop.Model.from_json` method that allows you to load a model from the json payload.
- SOP jobs can now be executed from the terminal by providing the path to the json file.

1.5.8 Release 1.8.11

Dependencies

- No new dependencies.

Bugs fixed

- Fixed a bug with the energy computation of the PO-Classical Optimizer.
- The Classical Optimizer returns a proper error message for large problems it cannot process.
- Improved documentation of the TN Optimizer parameters.

Features added

- A brand new TN Optimizer is now available using the optimizer name "tensor_network_experimental". This solver is much more stable and performant than "tensor_network" however it is still experimental meaning that we do not yet guarantee the absence of bugs. The solver was developed specifically for the use-case of portfolio optimization; therefore, while it is known to perform well on this problem, its performance on more general optimization problems is currently unknown.
- For environments that do not do SSL properly, users may disable checking. You can do this by setting the `ssl_verify` option to "False" in `singularity_config.json`, or by setting the environment variable `SINGULARITY_SSL_VERIFY` to `False`. The underlying python urllib` will log warnings of unverified SSL usage. For the best security adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/latest/advanced-usage.html#tls-warnings.`

1.5.9 Release 1.8.10

Dependencies

- No new dependencies.

Bugs fixed

- Multiverse hybrid sending unfeasible solution,

1.5.10 Release 1.8.9

Dependencies

- No new dependencies.

Bugs fixed

- TN Optimizer did not converge for small problems of integer variables, it has been corrected by using binary encoding by default. The docs regarding the encoder have been updated.
- Classical optimizer bug ignoring some input parameters has been patched.
- Exhaustive optimizer patched to not ignore variables in constraints that are not in the objective.

1.5.11 Release 1.8.8

Dependencies

- No new dependencies.

Features added

- NoSolutionError is a generic exception that solvers return when they cannot converge to a solution. If more detailed information is available it will be passed in a solver specific exception, e.g. SOPClassicalSolverError.

Bugs fixed

- exhaustive solver now returns feasible solutions.
- Fixed D-Wave problem formulation and cost comparison inside MultiverseHybridoptimiser.
- Change log included in the documentation.
- Return a ValueError when we fail to make a penalty out of a constraint.

1.5.12 Release 1.8.7

Dependencies

- No new dependencies.

Deprecated

REMINDERS

Features added

- Support of double bound inequality in Constraints.
- The default timeout updated to 30min.
- Extended to solver documentation.

Bugs fixed

- Datamodels type error fixed.
- Fixed encoder operation for variables.

1.5.13 Release 1.8.6

Dependencies

- No new dependencies.

Deprecated

REMINDERS

- Bring your own token is the default regarding DWAVE_API_TOKEN, please define yours.
- You should be using versions 1.8.x by now.

Features added

- Inequality precision per constraint: you can now specify the `inequality_precision` parameter per constraint (`sop.Constraint`). The default one being either `inequality_precision` you pass to `sop.Model.optimize()` or 2 in the backend.
- You can now add a single constraint without wrapping it up in a list. E.g. `sop.Model(objective, constraint)`.
- For people using more than one DWAVE_API_TOKEN, you can pass it as a parameter called `dwave_api_token` in the `sop.Model.optimize()` method.

Bugs fixed

- Inequality constraint rounding fixed for less than one coefficients (e.g. $1.24e-3$).
- Backend dependencies upgraded.

1.5.14 Release 1.8.5

Dependencies

- No new dependencies.

Deprecated

REMINDERS

- Bring your own token is the default regarding DWAVE_API_TOKEN, please define yours.
- You should be using versions 1.8.x by now.

Features added

- Document classical solvers `nlp_maximum_iterations`, this may help it converge to an optimum although it may not always converge.
- Improve error messages.
- Global timeout parameter: `sop.Model.optimize` has now a timeout parameter, if the computation runs more longer SOP will raise an error. Based on `asyncio`.
- Updated the Simulated Annealing solver from D-Wave dimod to D-Wave Neal which is faster and more flexible.
- Updated the backend to use `python3.10`.

Bugs fixed

- Binary encoder when preprocessing constraints.
- Casting `int/float` so that `sop.Model.eval_constraints` does not return false negatives.

1.5.15 Release 1.8.4

Incompatible changes

- BRING YOUR OWN TOKEN: mandatory, if you want to use D-Wave solvers you should have `DWAVE_API_TOKEN` set as an environment variable.

Bugs fixed

- Fixed docs.
- Error handling: improved messages when the D-Wave token runs out of credits.

1.5.16 Release 1.8.3

Incompatible changes

- The old backend is no longer available. You need frontends in the 1.8.x series or the 1.7.4 frontend with the environment variable `USE_SOP_NEW_BACKEND=true` set.
- **BRING YOUR OWN TOKEN:** mandatory, if you want to use dwave solvers you should have `DWAVE_API_TOKEN` set as an environment variable.

Dependencies

- The new backend does not use websockets, that dependency has been removed.

Deprecated

Features added

Bugs fixed

- Fixed docs.
- PO-like problem fixed, this bug occurred with the Multiverse Hybrid Optimizer when solving problems containing certain constraint boundaries.

1.5.17 Unsupported versions

1.5.18 Release 1.8.2

Dependencies

- No new dependencies in the client.

Incompatible changes

Deprecated

Features added

- Added ability to set inequality precision

Bugs fixed

- Fixed minor bugs

1.5.19 Release 1.8.1

Dependencies

Incompatible changes

Deprecated

- Clients prior to v1.7.4 are being deprecated and the corresponding backend will be eliminated soon (2.0.0 release). Please update to the newer versions $\geq 1.8.0$.

Features added

- singularity is now a namespace package, this means that you can install singularity.optimization together with other singularity.* packages that have been developed independently. For more information on namespace packages see <https://packaging.python.org/en/latest/guides/packaging-namespace-packages/#native-namespace-packages>.

Bugs fixed

- Bug 1: fix documentation regarding the deprecation of `Solution.qpu_readout`.
- Bug 2: a failure of the classical part of Multiverse Hybrid returned Unknown Error in the frontend; this has been debugged and the frontend gets more detailed error messages.

1.5.20 Release 1.8.0

Dependencies

- Requirements have not changed, but the new architecture does not require neither websockets nor nest-asyncio and they may be dropped in a future release.
 - symengine $\geq 0.9.0$
 - requests
 - websockets ≥ 10.1
 - nest-asyncio $\geq 1.5.4$

Incompatible changes

This release removes `qpu_readout` terminology in favor of the generic variables: there are a few classical solvers in Singularity Optimization, i.e. not all results are read from a QPU.

Deprecated

Versions prior to 1.7.4 use the old backend (slow, difficult to maintain). They will soon be deprecated.

Once the old backend is phased out, in order to keep using v1.7.4 you need to set `USE_SOP_NEW_BACKEND=true` in the environment.

If you want to use the old backend, you may set `USE_SOP_NEW_BACKEND=false` in your environment.

Features added

- Simulated Annealing: the simulated annealing is another classical solver you can use.
- Classical Solver: the regular classical solver now supports hundreds of variables, the limit being 1000 (17 previously).
- `DWAVE_TOKEN` can be sent as a parameter to the backend. I.e. BYOT, bring-you-own-token.

Bugs fixed

- Fixed docs.
- Backend tuning, the new architecture is now the default one.
- Some integer constraints previously generated a runtime error.

1.5.21 Release 1.7.4

Dependencies

- Requirements in the client have been relaxed to ease the integration of SOP python frontend as a dependency in another system.
 - `symengine` $\geq 0.9.0$
 - `requests`
 - `websockets` ≥ 10.1
 - `nest-asyncio` $\geq 1.5.4$

Incompatible changes

- EXPERIMENTAL: if you set the environment variable `USE_SOP_NEW_BACKEND=true` you will be directed to the new backend. This may reduce the latency to 1-2s.

Deprecated

Features added

- TN Optimizer: `svd_algorithm` is now a parameter of the TN Optimizer. Users may choose from:
 - full, the standard SVD algorithm in which all of the singular values are computed.
 - sparse, an iterative method which allows one to compute only a few singular values.
 - RSVD, randomized SVD, allows to compute only a few singular values by means of an intermediate random matrix.
- PO-Classical Optimizer: it now supports general (equality and inequality) linear constraints. Initially it only supported constraints of the type $w_i = K$, where w_i are the variables (portfolio weights) and K is an integer (total weight).

Bugs fixed

- Fixed docs.
- Backend tuning.

Due to incompatible changes in the backend versions prior to 1.7.4 are unsupported. Using 1.7.4 requires you to set the environment variable `USE_SOP_NEW_BACKEND` to true.

1.5.22 Release 1.7.3

Dependencies

- No new dependencies in the client.

Incompatible changes

Deprecated

Deprecated

Features added

Features added

- New order to read the token:
 1. environment variable
 2. working directory
 3. \$HOME directory
- ~20% faster networking.

Bugs fixed

Bugs fixed

- Reduced dependencies in network middleware

1.5.23 Release 1.7.2

Dependencies

Dependencies

- No new dependencies in the client.

Incompatible changes

Deprecated

Deprecated

Features added

Features added

- TN Optimizer: Adaptive Penalty Strengths.

Bugs fixed

Bugs fixed

- Minor bug fixies

1.5.24 Release 1.7.1

Dependencies

Dependencies

- No new dependencies in the client.

Incompatible changes

Deprecated

Deprecated

Features added

Features added

Bugs fixed

Bugs fixed

- Do not keep the web socket connection open indefinitely after receiving an error (in client).
- Fix treatment of binary variables in the classical solver (in backend).
- Accept linear problems in PO-Classical (in backend).
- Fix docs on PO-Classical and Multiverse-Hybrid solver (in client).

1.5.25 Release 1.7.0

Dependencies

Dependencies

- No new dependencies in the client.

Incompatible changes

Deprecated

Deprecated

Features added

Features added

- Classical solver based on GEKKO, the string in the client is classical.
- The Highspy based classical solver is now `po_classical`.

Bugs fixed

Bugs fixed

- Fix for linear problems submitted to classical solver.

1.5.26 Release 1.6.1

Dependencies

Dependencies No new dependencies.

Incompatible changes

Deprecated

Deprecated

Features added

Features added

- TN optimizer improvement: the Reduce-on-energy routine monitors the energy and reduces trotter_delta depending on whether the energy is stagnating.

Bugs fixed

Bugs fixed

- Increased concurrency in the backend by preventing cache collision.
- Fix memory leak when dealing with 10000s of constraints in the backend.
- TN: eliminate memory issue for integer variables.
- Properly document TN scheduler feature.

1.5.27 Release 1.6.0

Dependencies

Dependencies No new dependencies.

Incompatible changes

Deprecated

Deprecated

- The preferred way to use Singularity Optimization is by import singularity.optimization as sop and then using sop.Variable, sop.Model, sop.Objective. This way all things relative to sop live in a namespace separated from the working namespace, thus preventing name collisions and hard to catch bugs.

Features added

Features added

- Classical Solver: integer variables + convex problem solver.
- Multiverse Hybrid: for the same subset of problems of the Classical Solver, this optimizer combines the Classical Solver with the Dwave Solver. Very fast for Portfolio Optimization.
- TN optimizer now supports integer variables directly (faster!)
- A scheduler (adaptive_trotter) for the TN optimizer has been added: add hyperparameters `trotter_delta`, `trotter_steps` and `evolution_length` with `scheduler="adaptive_trotter"` to try.

Bugs fixed

Bugs fixed

- docs: replace ambiguous exhaustive with `dwave_exact` solver in basic usage.
- Reduced latency for TN, at least one GPU node always available for the TN optimizer.
- integer variables: fix bug when handling hundreds of variables with a range of more than a 10000.

1.5.28 Release 1.5.0

Dependencies

Dependencies No new dependencies.

Incompatible changes

Incompatible changes

- Client connection has been rearranged and it now tries to reconnect in the event of a network error.

Deprecated

Deprecated

Features added

Features added

- Integer variables. Users can now use bounded integer variables: from `singularity.optimization` import `Variable`, `INTEGER x = Variable(x, INTEGER, (27182,31415))`
- `singularity.optimization` now exposes `Variable`, `BINARY`, `INTEGER`, `Objective`, `Constraint`, `Model`, `OptimizationResult`, `Solution`.

Bugs fixed

Bugs fixed

- Reconnect attempts to the backend websocket prevent losing the optimization result on network error.
- The serialization format is fully compatible with QPBLIB.
- Fixed bug in TN docs.

1.5.29 Release 1.4.0

Dependencies

Dependencies No new dependencies.

Incompatible changes

- Server payload is now qplib-compatible, update client to use latest server.
- For variables the class is `singularity.optimization.Variable`, `singularity.optimization.modelling.types` is no longer supported.
- Note: the preferred/supported way to use `singularity.optimization` is by using the classes right under it: `Variable`, `Objective`, `Constraint`, `Model`.

Deprecated

Deprecated

Features added

Features added

- Symmetric Trotter Step in the TN optimizer.
- Serialization: more compact payload is now sent to the backend, anonymization is now free as no variable names are shared whatsoever.

Bugs fixed

- TN optimizer polish.
- Authentication error messages made more clear.

1.5.30 Release 1.3.0

@ Dependencies

Incompatible changes

- Update to the latest client due to incompatibilities with the logging feature

Deprecated

Deprecated

Features added

- Configurable TN timeout (setting `time_limit` in the solver parameters).
- Real time logging: the backend will send live messages via websocket to update the client on the optimizations progress.
- Polished TN optimizer.

Bugs fixed

1.5.31 Release 1.2.0

Dependencies

- The client works as usual, but now the backend uses GPUs to provide muscle for the Quantum Inspired TN Optimization.

Incompatible changes

Deprecated

Features added

- A new solver has been added for optimization: the TN (Tensor Network) Optimizer, the string is `tensor_network`.

Bugs fixed

- Set a timeout for long-running problems, instead of dragging on the client will rise an error on time out.
- Faster client loading (remove unnecessary imports).

1.5.32 Release 1.1.0

Features added

- Singularity can process now larger models
- The Singularity client is faster
- Information about solvers in the documentation
- Singularity supports inequality constraints

1.5.33 Release 1.0.3

Dependencies

Incompatible changes

Deprecated

Features added

Features added

- Enable DWaves Simulated Annealing solver
- Live documentation at <https://multiverse1.gitlab.io/product/optimisation/singularity-optimisation-client/>
- Slim down python library

Bugs fixed

1.5.34 Release 1.0.2

Dependencies

Incompatible changes

- The new backend provides a JSON object with fields {solutions, runtime, optimizer_info} all of which are available in the class `singularity.optimization.optimization.result.OptimizationResult`. Before it was an array with solutions, so older clients are not compatible with the new backend.

Deprecated

Features added

- Remove `qpu_state` from solution
- Add `runtime` and `optimiser_output` attributes to the solution
- Fix model validation taking too long
- Notify users when models size exceeds limit

Bugs fixed

- Wrong path for singularity_config.json
- Faulty expression validation

1.5.35 Release 1.0.1

Incompatible changes

The library now uses US spelling instead of the British one used previously in Singularity Products.

1.5.36 Release 1.0.0

The singularity optimization library is a very thin python client that allows you to formulate QUBO problems and solve them using various backends.

Features added

Note that the current release is not on par with the old singularity library yet. These are the features that we are currently supporting:

- create an optimization model using symbolic expressions
- add multiple equality constraints
- optimize using an exhaustive or the dwave solver

We plan to add more advanced features very shortly, such as inequality constraints. The inclusion of the TN optimizer is also being worked on. We will share a detailed roadmap in the coming weeks.

Acknowledgements

I would like to thank the product team for their hard work in the last months. They worked hard on building the infrastructure and foundations of the future Multiverse products!

1.6 License

IMPORTANT-READ CAREFULLY: This End User License Agreement (Agreement) is a legal agreement between you (either as a natural or legal person) and Multiverse Computing (hereinafter called the Company) for use of the Add-in (SOFTWARE). By accepting the terms and conditions of this Agreement and installing the Software or exercising your rights to make and use copies of the SOFTWARE (as may be provided for below), you agree to be bound by terms of this Agreement. If you do not agree to the terms of this Agreement, do not accept these terms and conditions and delete the SOFTWARE. You represent that you have full power, capacity and authority to enter into and accept the terms and conditions of this Agreement. If you are accepting on behalf of your employer, company or another entity, you warrant and represent that you have full legal authority to bind your employer, company or such entity to this Agreement, or that a person with such authority has accepted the terms and conditions of this Agreement prior to using the SOFTWARE as described in this paragraph.

1. All the rights for the SOFTWARE belong to Multiverse Computing.
2. The SOFTWARE is not free. The SOFTWARE is not sold, the SOFTWARE is licensed.

3. This license agreement, as well as the SOFTWARE use license, gives you non-exclusive right to use the SOFTWARE with the limitations this agreement explicitly provides for. User License Agreement
4. The license for use of the SOFTWARE gives you the non-exclusive right to install the SOFTWARE and use it on computers which number depends on the license type. If the SOFTWARE is installed in a local network for shared usage, the number of computers specified in the license should correspond to the number of the computers the SOFTWARE is launched on. License for use of the SOFTWARE doesn't impose any limitations on duration of the period of use (except for special limited trial use license)
5. The license for the SOFTWARE cannot be sold or transferred to third parties or leased without written permission of the copyright owner.
6. You may not, and you agree not to or enable others to, copy (except as expressly permitted by this License or by the Usage Rules if they are applicable to you), decompile, reverse engineer, disassemble, attempt to derive the source code of, decrypt, modify, or create derivative works of the Multiverse Computing software or any services provided by the Multiverse Computing Software, or any part thereof.
7. Multiverse Computing gives no warranty that the SOFTWARE meets your requirements. The purpose of the limited trial license is to give you an opportunity to evaluate whether the SOFTWARE meets your requirements.
8. The Software and documentation are provided AS IS without warranty of any kind. Multiverse Computing makes no warranties, expressed or implied, including, but not limited to, the implied warranties of fitness for a particular purpose or use. Multiverse Computing is not liable for the loss of profit or any other damage occurred due to your use or improper use of the SOFTWARE. You use the SOFTWARE at your own risk.
9. The software is subject to Spanish export laws and regulations. You must comply with all domestic and international export laws and regulations that apply to the software. These laws include restrictions on destinations, end users and end use.
10. Restrictions of usage of license: In particular, but without limitation, the licensed product may not be used, exported or re-exported
 - a. into any U.S., Canadian or European embargoed countries or listed on any sanctions list issued by Canada, USA or Spain.
 - b. the People's Republic of China, Russia and Iran, or
 - c. to any country, state or nation to which export or use is restricted under applicable Laws.
11. By using the licensed product, you represent and warrant that you are not located in any such country or on any such list. You also agree that you will not use these products for any purposes prohibited by United States, Canadian and Spanish Law, including, without limitation, the development, design, manufacture or production of missiles, or of nuclear, chemical, or biological weapons.
12. Multiverse Computing reserves all the rights not explicitly stated in the license.
13. Failure to comply with the provisions of this license agreement automatically results in termination of the SOFTWARE use license and may result in administrative and/or criminal prosecution.
14. Third Party Software. The Software may contain third party software that requires additional terms and conditions. Such required third party software notices and/or additional terms and conditions are located at https://cloud.dwavesys.com/leap/legal/cloud_subscription_agreement/ and are made a part of and incorporated by reference into this EULA. By accepting this EULA, you are also accepting the additional terms and conditions, if any, set forth therein.

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`